CPSC 320 Intermediate Algorithm Design and Analysis.
Professor: Abner Turkieltaub

Homework 2

This assignment is due **Friday, October 9 at 23:59 PM**. Late submissions will be accepted until Sunday October 11 at 23:59, but penalized: -5 points for Saturday and -15 for Sunday submissions. Please follow these guidelines:

- Prepare your solution using LaTeX and submit a pdf file. Easiest will be to submit using the .tex file provided. For questions where you need to select an option, simply change `\square` (□) to `\blacksquare` (■).
- Start your solution after our `\color{blue}` command. Your solution should appear in blue, making it a lot easier for TAs to find what you wrote.
- Start each problem on a new page, using the same numbering and ordering as this assignment handout. You can use `\newpage` to start a new page.
- Submit the assignment through GradeScope. Your group must make a **single** submission via one group member's account, marking all other group members in that submission **using GradeScope's interface**.
- When uploading to Gradescope, link each question with the page(s) of your pdf containing your solution.

Before we begin, a few notes on pseudocode throughout CPSC 320: Your pseudocode should communicate your algorithm clearly, concisely, correctly, and without irrelevant detail. Reasonable use of plain English is fine in such pseudocode. You should envision your audience as a capable CPSC 320 student unfamiliar with the problem you are solving. You may **neither** include what we consider to be irrelevant coding details **nor** assume that we understand any particular programming language[1].

Remember also to **justify/explain your answers**. We understand that gauging how much justification to provide can be tricky. Inevitably, judgment is applied by both student and grader as to how much is enough, or too much, and it's frustrating for all concerned when judgments don't align. Justifications/explanations need not be long or formal, but should be clear and specific (referring back to lines of pseudocode, for example). Proofs should be a bit more formal.

On the plus side, if you choose an incorrect answer when selecting an option but your reasoning shows partial understanding, you might get more marks than if no justification is provided. And the effort you put in writing down the justification will hopefully help you gain deeper understanding and may well help you converge to the right selection :).

Ok, time to get started...

[1] So, for example, do not write `#include <iostream>` at the start of your pseudocode, and avoid language-specific notation like C/C++/Java's ternary (question-mark-colon) operator.

1 Slow as a logarithm.

In "Algorithm Design" by Kleinberg and Tardos you can find (2.8, page 41) the following useful fact, stating roughly that logarithmic functions are upper-bounded by polynomial functions, more precisely:

Fact: for every $b > 1$ and every $x > 0$, we have $\log_b n \in O(n^x)$.

a) (10 pts.) Use the fact above to prove the following stronger assertion:

For every $b > 1$ and every $x > 0$, we have $\log_b n \in o(n^x)$.

b) (10 pts.) Use part a) to show that $\sqrt{n} \in o(n/\log^{12} n)$. The log is base 10, and $\log^k n = (\log n)^k$.

1. Slow as a logarithm.

a) To prove that $\log_b n \in o(n^x)$, we need to prove that the $\lim_{n\to\infty} \frac{\log_b n}{n^x} = 0$. And for that, we can leverage the Squeeze Theorem. For the Squeeze Theorem, we need to establish a lower bound and an upper bound. For the lower bound, we will just use 0 (since we want to prove the limit equals 0). For the upper bound, it can be conveniently derived by the fact stated in the question, along with the condition $x > 0$. The fact states that $\log_b n \in O(n^x)$, meaning $\log_b n \leq c \cdot n^x$ for some constant $c > 0$. Since $x > 0$ we can assume that $x \in \mathbb{R}_{>0}$, with $\mathbb{R}_{>0}$ being an uncountable set. This is an important fact because then we know that $\forall x \in \mathbb{R}_{>0}, \exists \alpha \in \mathbb{R}_{>0} : 0 < \alpha < x$ (which is just a fancy way of saying for every $x > 0$, there is some value α that is smaller than x but still greater than 0). So, we can set up the Squeeze Theorem as such:

$$\lim_{n\to\infty} 0 \leq \frac{\log_b n}{n^x} \leq \frac{c \cdot n^\alpha}{n^x}$$

$$= \lim_{n\to\infty} 0 \leq \frac{\log_b n}{n^x} \leq \frac{c}{n^{x-\alpha}}$$

Since we know that c is constant, even if $x - \alpha$ is tiny, as n goes to ∞, the denominator will still get really large. Therefore:

$$= \lim_{n\to\infty} 0 \leq \frac{\log_b n}{n^x} \leq 0$$

By the Squeeze Theorem, since the lower bound and the upper bound both converge to 0, the middle term ($\frac{\log_b n}{n^x}$) must equal 0.

$$\lim_{n\to\infty} \frac{\log_b n}{n^x} = 0$$

b) To show that $\sqrt{n} \in o(n/\log^{12} n)$, we need to show that the $\lim_{n\to\infty} \frac{\sqrt{n}}{\frac{n}{\log^{12} n}} = 0$. First step in showing that would be to clean up the fraction.

$$\lim_{n\to\infty} \frac{\sqrt{n}}{\frac{n}{\log^{12} n}} = \lim_{n\to\infty} \frac{\log^{12} n}{\sqrt{n}} = \lim_{n\to\infty} \frac{(\log n)^{12}}{\sqrt{n}}$$

Now, to use part a) in the solution, we want to convert that fraction into the form $\frac{\log_b n}{n^x}$.

$$\lim_{n\to\infty} \frac{(\log n)^{12}}{\sqrt{n}} = \lim_{n\to\infty} \frac{(\log n)^{12}}{n^{12x}} = \lim_{n\to\infty} \left(\frac{\log n}{n^x}\right)^{12}$$

For the above equalities, we need to make sure that $\sqrt{n} = n^{12x}$ for some $x > 0$. In this case, we want $x = \frac{1}{24}$. We can then rewrite it with x filled in:

$$\lim_{n\to\infty} \left(\frac{\log n}{n^{\frac{1}{24}}}\right)^{12}$$

By part a), since the limit above satisfies the form $\frac{\log_b n}{n^x}$ and $\frac{1}{24} > 0$, we know that the term in the base goes towards 0 as $n \to \infty$.

$$\lim_{n\to\infty} \left(\frac{\log n}{n^{\frac{1}{24}}}\right)^{12} = \lim_{n\to\infty} \frac{\log n}{n^{\frac{1}{24}}} = 0$$

2 I wanna be the very best

In the game "Pokemon Go", there is a pvp[2] feature. In this feature, each player selects a team of 3 different pokemons to fight a match against another player. Assume that each pokemon can have one of three roles: "Lead", "Safe swap" or "Closer". We say that a team is *balanced* if it has the three roles[3].

a) (5 pts.) We say that a valid team is just a set of three different pokemons. If you have n pokemons to choose from, how many valid teams are there?

b) (10 pts.) We want to print a list with all the balanced teams. Consider a brute force approach where you look at every valid team and you print it if it is balanced. Write a pseudo-code to implement this idea using a nested for loop structure, and find a tight O-bound for the worst case runtime. Your input is an array of pokemon $[p_1, p_2, \ldots, p_n]$ and you can query the role of any pokemon using p_i.role() in constant time. Justify your runtime.

c) (5 pts.) Usually brute force is too slow, but in this case it is not that bad. Assuming that printing one team takes constant time, show that there are instances for which any algorithm that must print all balanced teams has $\Omega(n^3)$ runtime.

d) (5 pts.) Define the following sets:

$$L = \{p_i : p_i.\text{role}() = \text{Lead}\}$$
$$S = \{p_i : p_i.\text{role}() = \text{Safe Swap}\}$$
$$C = \{p_i : p_i.\text{role}() = \text{Closer}\}$$

Write the number of balanced teams as a function of $|L|,\ |S|,\ |R|$.

e) (10 pts.) In general, you express the runtime as a function of the input size. Sometimes you can analyze the runtime in terms of other parameters to have a more detailed answer. First analyze the runtime of your previous brute force approach in terms of $|L|,\ |S|$ and $|R|$. Then describe a more efficient algorithm in terms of $|L|,\ |S|$ and $|R|$. For your second algorithm, what is the best and worst possible runtime in terms of n?

[2] Player vs player.

[3] In case you are curious, you start a match with 2 shields that can be used to block attacks. The lead should be strong at the beginning of the match when both players have shields. The save swap should be versatile and have few hard counters, so when the lead is unfavourable, you can switch to it. The closer is strong at the end of the game when shields are down. All of this is irrelevant to answer the question.

2. I wanna be the very best

a) If we say that a valid team is a **set** of 3 different pokemons, then the number of valid teams would be $\binom{n}{3}$. By definition, this is equal to $\frac{(n)_3}{3!} = \frac{n \cdot (n-1) \cdot (n-2)}{3!}$. In this case you would divide by 3! to account for double counting (i.e. order does not matter). So, with $\binom{n}{3}$, we would consider teams (a, b, c) and (a, c, b) to be equal.

b) The brute force approach is unaware of any pokemon roles, so you would need to first gather all the valid teams (set of size $\binom{n}{3}$), then from those pick out the ones that are balanced. In order to find all the valid teams, it would involve a triple-nested loop, where the inner loop moves the fastest, and every nested loop depends on the loop that contains it. For filtering, there are a couple options, but the easiest way to check would just be if all of them have distinct roles. No loop, accumulator, or boolean flags would be required. Since there are 3 slots and 3 roles, we know that if they are all distinct, the team must be balanced. Therefore, we would choose to print it. The pseudocode would look something like this:

Algorithm 1 Brute Force Balanced Teams

```
1:  procedure PrintBalancedTeams(P)
2:      teams = []
3:      for each pokemon i in P do
4:          for each pokemon j = i+1 to n do
5:              for each pokemon k = j+1 to n do
6:                  Append (i, j, k) to teams
7:              end for
8:          end for
9:      end for
10:     for each team t in teams do
11:         if Pokemon 1 Role ≠ Pokemon 2 Role ≠ Pokemon 3 Role then
12:             Print t
13:         end if
14:     end for
15: end procedure
```

To analyze the runtime, we want to look at the triple-nested for loop and the filtering for loop. For the triple-nested for loop, since each of the inner loops depend on the one that contains it (e.g. j = i + 1), the runtime would follow a summation pattern for every set of nested loops. So if we just look at the loops corresponding to pokemon j and pokemon k, we would get a runtime of:

$$\sum_{j=i+1}^{n} (n - j)$$

That sum describes the runtime of the inner two loops based on the value of i. Then, to account for the outermost loop, you would just sum the runtimes from $i = 1$ to $i = n$.

$$\sum_{i=1}^{n} \sum_{j=i+1}^{n} (n - j)$$

Now, to figure out the approximate bound of that summation, we're going to need some clever substitutions and the summation formula for i and i^2.

Let $n - j = m$

$$\text{Upper bound: } n-(i+1) = n-i-1$$

$$\text{Lower bound: } n-n = 0$$

$$\sum_{j=i+1}^{n}(n-j) = \sum_{m=0}^{n-i-1} m = \frac{(n-i-1)(n-i)}{2}$$

$$\sum_{i=1}^{n}\sum_{j=i+1}^{n}(n-j) = \sum_{i=1}^{n}\frac{(n-i-1)(n-i)}{2}$$

$$\text{Let } k = n-i$$

$$\sum_{i=1}^{n}\sum_{j=i+1}^{n}(n-j) = \sum_{i=1}^{n}\frac{(k-1)k}{2} = \sum_{i=1}^{n}\frac{k^2-k}{2} = \frac{1}{2}\sum_{i=1}^{n}(k^2-k)$$

$$\frac{1}{2}\left(\sum_{i=1}^{n}k^2 - \sum_{i=1}^{n}k\right) = \frac{1}{2}\left(\frac{k(k+1)(2k+1)}{6} - \frac{k(k+1)}{2}\right)$$

So, with the calculations above, we can conclude that the tight O-bound for the triple-nested loop is $O(n^3)$. For the filtering for loop, it is straightforward since we already know how many valid teams exist from part a). That loop would run $\binom{n}{3}$ times, which has a tight O-bound of $O(n^3)$. So with both those in mind, the tight O-bound of the entire algorithm is $O(n^3)$.

c) An instance where regardless of the algorithm, it would print all the balanced team in $\Omega(n^3)$ runtime would be the case where all role sets have size $\frac{n}{3}$. In the case of brute force, it would run in $\Omega(n^3)$ regardless of the shape of the input. This is because you don't know each pokemon's role, meaning you would have to check every pokemon against every other pokemon. In the case of a more efficient algorithm where you already know all pokemons' roles, it still would run in $\Omega(n^3)$ with this particular instance. This is because for each of the $\frac{n}{3}$ leads, you would have to check it against the $\frac{n}{3}$ safe swaps, and the $\frac{n}{3}$ closers. This would run in $\Omega(\frac{n^3}{27}) \in \Omega(n^3)$.

d) With all the roles segregated into sets, the number of balanced teams would simply be the product of all the set sizes ($|L||S||C|$). In this case, we wouldn't have to divide by 3! like we did in part a) because of the fact that $|L|$, $|S|$, and $|C|$ disjoint sets (we can safely assume this from the question "each pokemon can have one of three roles"). Therefore, for all the slots, you wouldn't have to worry about a closer being in the lead spot or something of that nature. All that's to say that we would not have to account for double counting.

e) Analyzing the runtime of the brute force approach in terms of $|L|$, $|S|$, and $|C|$, we would conclude that the runtime is $O((|L|+|S|+|C|)^3)$. This is just equivalent to $O(n^3)$ since we know that all sets are disjoint. The efficient way to print out all balanced teams would first start with segregating the pokemons by role. So, we would need to iterate through the list 1 time. Now that we have all our lead, safe swap, and closer pokemons, we would have 3 for loops nested inside one another, where for every lead, it would iterate through all safe swaps, which would iterate through all closers. For every closer, we would print the associated balanced team. So, the pseudocode would look something like this:

Algorithm 2 Efficient Balanced Teams

```
1: procedure PrintBalancedTeams(P)
2:     lead = []
3:     swap = []
4:     closer = []
5:     for each pokemon p in P do
6:         if p.role() = Lead then
7:             Append p to lead
8:         else if p.role() = Safe Swap then
9:             Append p to swap
10:        else
11:            Append p to closer
12:        end if
13:    end for
14:    for each pokemon l in lead do
15:        for each pokemon s in swap do
16:            for each pokemon c in closer do
17:                Print (l,s,c)
18:            end for
19:        end for
20:    end for
21: end procedure
```

For this second algorithm, the best case runtime would be $O(n)$. This is because the initial for loop to segregate by role would be $O(n)$, then for the triple-nested for loop, an input with something like 1 lead, 1 safe swap, and $n-2$ closers would allow it to run in $O(n)$. We can safely assume this because we know that checking role and printing are constant operations. And only the inner most loop would run $n-2$ times which is $O(n)$. Now for the worst case runtime, it would be $O(n^3)$. This would come as a result of an input like all sets being of size $\frac{n}{3}$. The segregation would still be $O(n)$, but the triple-nested for loop would instead be $O(\frac{n^3}{27}) \in O(n^3)$. Since the n^3 dominates, the worst case would be $O(n^3)$.

3 Stirling

The Stirling approximation is a famous and useful formula that approximates the factorial. One way to express it is as follows:

$$\ln(n!) = n \ln n - n + O(\ln n).$$

In other words, $\ln(n!)$ is approximately equal to $n \ln n - n$ plus something bounded by a logarithmic term. In particular, we can say that $\ln(n!) \in \Theta(n \ln n)$. In this question we will prove this simplified approximation (without using Stirling).

a) (5 pts.) Prove that $\ln(n!) \in O(n \ln n)$.

b) (10 pts.) Using induction, prove that $(n!)^2 \geq n^n \ \forall n \in \mathbb{N}$.
Hint: it may be useful to prove by induction the following property first: $\forall n \in \mathbb{N},\ n^n \geq (n+1)^{n-1}$.

c) (10 pts.) Use the previous result to conclude that $\ln(n!) \in \Theta(n \ln n)$

d) (5 pts.) Either prove or provide a counterexample for the following statement: if h is an increasing function and $h(f(n)) \in \Theta(h(g(n)))$, then $f(n) \in \Theta(g(n))$.

3. Stirling

a) To prove that $\ln(n!) \in O(n \ln n)$, we need to show that $\ln(n!) \leq c \cdot n \ln n$ for some constant $c > 0$. First, through the use of logarithm laws we know that $\ln(a^n) = n \ln(a)$. So, we can rewrite $n \ln n$ as $\ln(n^n)$. So, we would then be left with this inequality to prove:

$$\ln(n!) \leq \ln(n^n)$$

Now, for strictly increasing functions, we can use the property of strict monotonicity and exponentiate both sides with base e to get the same inequality but with only the terms within the parentheses (i.e. $n!$ and n^n). Now, we can show that $\ln x$ is strictly increasing over its domain by taking the first derivative and checking if it is always positive (> 0).

$$\frac{dy}{dx} \ln x = \frac{1}{x}$$

$$\forall x > 0, \ \frac{1}{x} > 0$$

$$\therefore n! \leq n^n$$

Now, the inequality above is much nicer to prove. And for this question we are assuming that $n \geq 0$ since factorials are only defined on non-negative integers. For this proof we are going to expand both terms and compare factors.

$$n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 1 \leq n \cdot n \cdot n \cdot \ldots \cdot n$$

$$(n-1) < n, \ (n-2) < n, \ \ldots, \ 1 < n$$

Since we established the domain being only non-negative integers, we can safely say that comparing individual factors like that will hold. Therefore $\ln(n!) \leq \ln(n^n) = \ln(n!) \leq c \cdot n \ln n$, meaning $\ln(n!) \in O(n \ln n)$.

b)

c)

d) For h, we can use $\ln x$ as shown in part a) as a strictly increasing function over its domain. Now, a counterexample would be if you defined $f(n) = n^3$ and $g(n) = n^2$.

$$h(f(n)) \in \Theta(h(g(n)))$$

$$h(f(n)) \leq c \cdot h(g(n)) \text{ and } h(f(n)) \geq c \cdot h(g(n))$$

$$h(n^3) \leq c \cdot h(n^2) \text{ and } h(n^3) \geq c \cdot h(n^2)$$

$$\ln(n^3) \leq c \cdot \ln(n^2) \text{ and } \ln(n^3) \geq c \cdot \ln(n^2)$$

By the logarithm law shown in part a), the inequalities above can be rewritten as such:

$$3 \ln(n) \leq c \cdot 2 \ln(n) \text{ and } 3 \ln(n) \geq c \cdot 2 \ln(n)$$

Since with O and Ω we can disregard constants, we can rewrite it as:

$$\ln(n) \leq \ln(n) \text{ and } \ln(n) \geq \ln(n)$$

This means that $h(f(n)) \in \Theta(h(g(n)))$. But, for $f(n) \in \Theta(g(n))$ on the other hand, the inequalities would look like this:

$$n^3 \leq c \cdot n^2 \text{ and } n^3 \geq c \cdot n^2$$

We can see that for the inequalities above only the right side holds (i.e. $f(n) \in \Omega(g(n))$). That means that $f(n) \notin \Theta(g(n))$.

4 It's not the destination, it's the journey.

In planet Namek, all the alien kids have now their school for the upcoming school year. For parents, the question now is what is the fastest way to get to school. For parents with only one kid this is trivial since they can use noodle maps to find the shortest path from their home to the school[4]. Consider a parent of twins in different schools, this parent wants to find the shortest route to drop both kids and then return home[5].

a) (5 pts.) For each of the following scenarios, explain how they can find the shortest route using noodle maps the least number of times possible.

- Case 1: distances are symmetric (distance from A to B is the same as the distance from B to A).
- Case 2: distances are asymmetric[6].

b) (5 pts.) Many algorithms for shortest path (like Dijkstra) actually find the shortest path from one vertex to all the others. Imagine you are a developer for noodle maps and you want to implement a query where the user (a parent) inputs their home and the two schools and it shows them the shortest route starting from their home, passing by the two schools and returning home. Assume that as a developer, when you run noodle maps to find a shortest path from A to B, you can actually access the shortest paths from A to every other location. Answer the questions from the previous part under this new hypothesis.

c) (5 pts.) Answer the questions from the previous two parts, but now under the assumption that work from home is not allowed, so parents must go to their work after dropping both kids.

[4] In noodle map you can enter two locations A and B and it tells you the shortest path from A to B.

[5] In Namek, all parent can work from home.

[6] Maybe there's a school in the top of a mountain and you have to take some long stairs to get there, but then when you leave you can go down a slide.

Statement on Collaboration and Use of Resources

To develop good practices in doing homeworks, citing resources and acknowledging input from others, please complete the following.

None of your answers here will be used against you, so please be honest. Not answering this question will result in a deduction of 2 points.

1. Please list the CWLs of all group members.
   Andrew Li - andrwhli

2. List here any books or online sources that you consulted and that led you to the solution of one or more problems (indicate the problem(s) after each resource):
   `https://en.wikipedia.org/wiki/List_of_logarithmic_identities` - Problem 1
   `https://en.wikipedia.org/wiki/Squeeze_theorem` - Problem 1
   `https://en.wikipedia.org/wiki/Summation` - Problem 2

3. One or more of us consulted with course staff during office hours:
   ☐ Yes ■ No

4. One or more of us collaborated with other CPSC 320 students outside our group:
   ☐ Yes ■ No
   If yes, please list their name(s) here:

5. One or more of us used an AI to solve one or more questions from this assignment:
   ☐ Yes ☐ No
   If yes, please say which problem(s) was(were) solved using AI and which AI model you used:

6. One or more of us collaborated with or consulted others outside of CPSC 320:
   ☐ Yes ■ No
   If yes, please list their name(s) here: